CodeLLM Part 6：Gemma 4 SWE Agent 的 scaffold 工程与优化记录（大白话版）

Bo Hou（与 Claude 协作整理） · 2026-10

Kaggle「Google – Gemma 4 Developer Agent」第一轮：本地 129 题从 51 做到 58，隐藏集首次提交 0.13（当时榜首 0.24）

目录

0. 这篇讲什么、怎么读
1. 起点：比赛要什么、我们手上有什么
2. 第一步：先把测试集弄对
3. 基线 sub_v2：先跑通，再看分
4. 5 分钟预算下的第一轮迭代（v3–v8）
5. 拆轨迹：时间到底花在哪
6. Bo 的方向：CPU 做查找和排序，LLM 只做判断
7. 通宵自动搜索：v9–v14
8. 这个分数可信吗：给 v9 的 58/129 做体检
9. 打包、算时间、提交 Kaggle
10. 合作方式上的几点
11. 现在在哪、接下来
12. 附录：版本一览、关键数字、术语、链接

0 这篇讲什么、怎么读

Part 5 讲的是"这个比赛是什么、SWE agent 和 RL 能怎么做、别人怎么做"。这篇 Part 6 讲的是动手的部分：从环境搭起来，到第一次在 Kaggle 上拿到分数，中间每一步做了什么、为什么这么做、踩了哪些坑、哪些想法被数据否掉了。

一句话结论：**scaffold（给模型配的提示词、工具、预算）这一轮基本榨干了**。本地 129 题从 51 题做到 58 题，而且每题的时间从平均约 9 分钟压到了 4 分钟以内。但到了 Kaggle 隐藏集上只有 0.13。原因主要有两个：本地题模型很多见过，隐藏题更新更难。下一步的重点是造一套更像隐藏集的新题，再在上面做 RL。

文中用几个标签区分"谁定的、谁做的"：

标签	意思
Bo 定的	方向、规矩、取舍，是 Bo 拍板的。
Claude 的优化	Claude 做的分析、设计和实现，后面会写"为什么"。
Chrome 执行	Claude for Chrome 在 GPU 机器上实际动手做的：改文件、跑实验、汇报结果。
Claude 的错误	Claude 判断错、写错的地方，直接写出来。
教训	事后回头看，值得记住的东西。

分工：Bo 在 VS Code 和浏览器里操作，做决定，也做 review。这个对话里的 Claude 负责分析数据、设计方案、写给 Chrome 的指令。Claude for Chrome 负责在 Lambda GPU 机器上执行，比如改 submission、跑评测、写日志。

1 起点：比赛要什么、我们手上有什么

1.1 一句话目标

用主办方指定的 `gemma-4-31b-it-qat-w4a16-ct`（31B，4 bit 量化，只能用这一个基座），做一个能自己修 bug 的 agent。它要做的事：读题目（通常是一个 GitHub issue 或 PR 描述），在 Python 仓库里找到问题，改代码，交出补丁。判分方法是用隐藏的测试去跑这个补丁，测试通过就算这题做对。指标是做对的比例（Resolution Rate）。

提交的东西不是 Python 代码，而是一个"声明式"的 agent 配置，内容包括：`agent.yaml`（用哪些工具、用哪个提示词）、提示词文件、采样参数、预算，以及可选的 skill 脚本和 LoRA。主办方的编译器把它编译成一个 Google ADK agent 来跑。

1.2 评测怎么跑：两个容器

阶段	做什么
容器 A（agent）	agent 在 `/workspace` 里用工具看代码、跑命令、改代码。结束时，harness 用 `git diff` 把改动取成补丁。
容器 B（判分）	打上 agent 的补丁，把被改过的测试文件重置回原样，再打上隐藏的 test_patch，然后跑 pytest。指定的测试全部通过才算对。

可以用的工具有 9 个：`run_command`、`read_file`、`edit_file`、`write_file`、`get_status`、`submit_patch`，还有 3 个代码图工具。上下文最长 32K token，带自动压缩（把旧对话总结掉）。

1.3 一道题长什么样：以 fastapi_11194 为例

① **agent 收到的题目**，是一段 PR 描述，简化后是：

> 🐛 修复：当 `File` 参数写在 `Form` 参数后面时，接口会返回 422 校验错误。现在必须把 `File` 写在前面，否则请求失败。

除了题目，agent 还会收到预算（几分钟、几次调用）、环境规则（离线、每次最多读 150 行）和仓库的目录树。仓库放在 `/workspace`，是这个 PR 合并之前的代码。

② **agent 要做的**：找到 FastAPI 解析表单参数的代码，修好"参数顺序一换就出错"的问题，然后交出补丁。注意这里的 `File` 不是文件名，而是 FastAPI 里的一个参数类型（`File()`，它的父类是 `Form()`），所以要在代码里搜这个名字。v7 的找法是这样的：先在 /tmp 写个小脚本，试着复现问题；然后 `grep "class Form"` 和 `grep "class File"`，找到 `fastapi/params.py`；读了之后发现那里只是定义，问题不在这；再接着找"表单参数在哪里被解析"，最后在 `fastapi/dependencies/utils.py` 里修改。光是从一个名字找到定义、再从定义找到真正用它的地方，就花了好几轮。

③ **判分时发生什么**：harness 先打上 agent 的补丁，再打上这个 PR 附带的隐藏测试，然后跑 pytest。隐藏测试大概是这样：

```
@app.post("/upload")
def upload(name: str = Form(), file: UploadFile = File()): ...

resp = client.post("/upload", data=..., files=...)
assert resp.status_code == 200     # 没修好时报：assert 422 == 200
```

agent 看不到隐藏测试，只能根据题目去猜测试会检查什么。所以有些题会因为措辞或细节不一致而判错，比如测试要求的报错原文，题目里根本没给。

1.4 硬件和成本

	本地（我们的）	Kaggle 判分机
GPU	Lambda A100-40GB，单卡，$1.99/小时	4 × L4（TP=4）
推理	vLLM，32K 上下文，3 个 worker 同时跑 3 道题	vLLM，题目一道一道顺序跑
时间限制	自己定	整个提交最多 12 小时（隐藏集约 120 题）

到第一次提交为止，Lambda 一共花了约 $280。机器按开着的小时数计费，GPU 空闲也照样收钱，所以后面改成：不跑的时候就 terminate，分析和 debug 搬到 CPU 机器上做。

1.5 总方针

> **Bo 定的**
>
> 1. **测试集是第一步，必须弄对。**判分不准，后面所有比较都没有意义。
> 2. **先把 scaffold 榨干，再考虑 RL。**原话："不着急，先看 scaffold，不能再提升或者榨干之后再说。另外我人也要分析和看。"
> 3. **能让 CPU 做的，就不浪费 LLM 的推理。**原话："能跑程序 CPU 去找 code repository bug 的，就不要浪费 LLM 的推理能力。"
> 4. **每一步都要让人看得懂。**要说人话，抽象层次要合适，不能让人迷路。

2 第一步：先把测试集弄对

2.1 为什么先做这个

主办方给了 129 道公开题，分属 4 个仓库：fastapi、rich、requests、httpx。我们自己判分，用的是本地复刻的 harness 和 docker 沙箱。如果环境本身就有问题（比如缺依赖、版本不对、需要联网），那么就算补丁是对的，也会被判错。这样拿来比较版本，等于用一把歪尺子量东西。

2.2 方法：用标准答案（gold patch）检验环境

思路很简单：把每题的标准答案补丁直接当成 agent 的输出拿去判分。标准答案都过不了，说明问题出在环境，不在 agent。第一轮检验，129 题里有 108 题通过，**21 题**连标准答案都过不了。

2.3 21 道判不了的题，按原因分组修

原因	题数	症状	怎么修的
anyio 插件	3	判分命令里写死了 `-p no:anyio`，这几道 fastapi 的异步测试跑不起来	给 harness 加了一个环境变量开关，只在这 3 题上改成 `-p anyio`，默认行为不变
需要 httpbin	8	requests 的测试要起一个本地 HTTP 服务（pytest-httpbin），沙箱里没有	做了一个装好 httpbin 和对应 werkzeug 版本的镜像。另外建了一个"tarpit"网络：容器可以连自己本机的服务，但出不了外网，宿主机也用 iptables 挡住
pygments 版本	7	rich 的测试期望的高亮输出和 pygments 版本有关	每题搜一个能让标准答案通过的 pygments 版本，钉死（分别是 2.13.0、2.16.1、2.19.2）
Python 版本	1	rich_3472 在 Python 3.13 下行为不同	换成 3.12 的镜像
harness 把 skip 当失败	2	rich_3486、fastapi_14186 里有按 Python 版本跳过的测试，harness 把"跳过"判成了"失败"	加环境变量 `SWG_SKIP_AS_PASS`，只对这 2 题生效
依赖包污染	—	fastapi_14356 失败（见 2.4 的错误）	清理 wheel 目录，恢复主办方原始的那一套
依赖时间	1	requests_6757 的测试证书有过期时间，现在跑就过期了	只在这一题的判分容器里用 faketime 把时间往回拨 250 天（Bo 单独授权，只限这一题）

修完以后：**129/129 全部可以判分**。默认环境下 108 题直接通过，另外 21 题通过 `overrides.json` 给每题单独指定环境，回归检验 22/22 通过。这些修改全部用环境变量控制，默认行为不变，所以不会影响其他题。

Bo 定的 修环境的活，Bo 让 Chrome 自己去改，原话是"直到剩下的失败 fix"。rich_3486 那题，Bo 指出是 harness 的 bug，原话："harness bug，不影响咱们用，而且一个环境变量也没啥。" 所以用环境变量绕过去，不当成题目本身的缺陷。

2.4 这一段里 Claude 犯的错

Claude 的错误

1. **fastapi_14356 找错原因。**Claude 先说是 starlette 版本的问题，后来查证真正的原因是 Claude 自己写的环境修复脚本有 bug：一个 `tr` 命令用错了，多装进了 45 个 wheel，其中 brotlicffi 改变了响应压缩的行为。清掉多装的 wheel，这题就好了。
2. **requests 题说错了范围。**Claude 说"超时相关的测试不是必须通过的"。实际上 harness 要求整个测试文件全部通过。
3. **rich_3486 定性错了。**Claude 说是"题目本身有缺陷"。Bo 追问"不是说根据 Python 版本判断吗？pytest 要求这两个都要过？"，这才查清楚：问题出在 harness 把 skip 当成了失败。
4. **差点覆盖掉已有配置。**一个修复脚本的第二版（fix20b）如果直接跑，会把已经调好的 pygments 版本设置全部清掉。第三版改成只合并、不覆盖。另外，查找文件时用的通配符区分大小写，漏掉了 `Pygments-2.16.1` 这种首字母大写的文件名，改成了 `[Pp]ygments`。

教训

1. "标准答案都过不了"是检验判分环境最便宜、最可靠的办法，应该第一个做。
2. 每个修复只对指定的题生效，默认行为不动，这样修一题不会把别的题弄坏。
3. AI 给出的原因不能直接相信，要看证据，也要有人 review。这一段四个错里有三个是 Bo 追问出来的。

3 基线 sub_v2：先跑通，再看分

3.1 跑通之前踩的坑

现象	原因	怎么改的
对话跑几轮就超出 32K 上下文	本地 CLI 没有打开 Kaggle 上默认开启的事件压缩	写了一个 `swegemma_local.py` 包装脚本，按 Kaggle 的参数打开压缩（每 5 条压一次，阈值 14336 token）
import 报错缺 typing_inspection	离线 wheel 目录里少了这个包	补上 wheel，清缓存
fastapi 的题"太容易"	wheel 目录里有 fastapi 新版本的包，等于把答案放进了环境	删掉，每个仓库的 wheel 目录都排除仓库自己
一题调用了 470 次 submit	harness 只有在 agent 回复纯文字之后才结束，模型交完补丁又接着交	提示词里加一条"交完说一句话就停"（Stop clause），再加上轮数上限
一整晚的结果没了	批量脚本用的 venv 里没装 kaggle 包，一启动就失败了	在 venv 里补装 kaggle

3.2 结果：51/129（每题给 60 分钟、100 次调用）

sub_v2 的配置：主 agent 加一个"侦察"子 agent（investigator），提示词里写了 Stop clause。78 道失败题按主要原因分：

主要原因	题数	大白话
WRONG_BEHAVIOR:logic	20	改的地方对，逻辑写错了
TOUCHED_TESTS	17	agent 改了测试文件，导致隐藏测试补丁打不上去，整题判错
EMPTY/BUDGET	14	时间或调用次数用完了，一行都没改
WRONG_FILES	11	改错了文件
MISSING_NEW_FILE	6	题目需要新建一个文件，agent 没建
WRONG_BEHAVIOR:message	6	报错信息的文字和测试期望的不一致
REGRESSION	4	修好了这个问题，又弄坏了别的

3.3 按年份看：一个后来被证实的担心

按题目对应 PR 的年份统计：2026 年的题通过率约 30%，更早的题约 48%（$p \approx 0.06$）。这说明模型很可能在训练时见过老题。Bo 当时的原话："可问题是 Gemma 给的模型还都见过，题目出得……"。第 8 节和第 9 节会看到，这个担心是对的。

4 5 分钟预算下的第一轮迭代（v3–v8）

4.1 为什么要压到 5 分钟

Kaggle 整个提交只给 12 小时，隐藏集大约 120 题，平均每题 6 分钟，这里面还要算上容器启动等开销。sub_v2 每题平均 8.8 分钟，到 Kaggle 上会超时，必须压下来。

> **Bo 定的** "5 分钟挺好的，这个是大 improvement。" 意思是：每题 5 分钟是硬约束，先在这个约束下把分数做上去。

4.2 怎么迭代：开发集和噪声

每改一版都跑全部 129 题太慢。Chrome 从中挑了 43 题作为**开发集**：sub_v2 里改了测试文件的 12 题、空补丁或超预算的 14 题、改错的 10 题，再加 8 道 sub_v2 做对的题当对照组。每次只改一个地方，比较开发集上的分数。噪声大约是 ±2 到 4 题，差别在这个范围内的不算数。

4.3 每一版改了什么、结果如何

版本	改了什么	开发集/43	结论
v2	参照：60 分钟、100 次调用	8	开发集是从它的失败题里挑的，所以分数偏低是必然的，不能拿来直接比
v3	只把预算改成 5 分钟、60 次调用	10	20 题交了空补丁：5 分钟基本都耗在 investigator 子 agent 上了
v4	提示词加"时间预算"一节：一半时间之前必须开始改代码	13	空补丁从 20 降到 7。模型几乎不调用 get_status 看时间
v5	去掉 investigator 子 agent	13	分数没变，但结构更简单，保留
v6	保护测试文件：不许碰 tests/；提交前用 `git status` 检查	17	改测试文件导致的失败从 5 降到 0，是真实提升
v7	工具调用上限从 60 改成 30	17	分数没变，每题时间中位数从 308 秒降到 221 秒，保留
v8	改完代码**必须**跑相关的回归测试	16	没有提升，空补丁反而变多了：测试吃掉了调用次数。不采用

教训

1. 在 5 分钟里，**任何多加的步骤都会挤掉"改代码"的时间**。v8 的教训是：验证必须是可选的，而且要便宜。
2. 分数的提升主要来自"不犯低级错误"，比如别碰测试文件、别把时间花在子 agent 上，而不是模型变聪明了。
3. Bo 问过："v2 是 51/129，v3 到 v8 怎么只提升了 3 道？" 答案是：开发集专门挑的是 v2 的失败题，两者不能直接比。开发集只能用来比较 v3 及之后的版本。

5 拆轨迹：时间到底花在哪

Claude 的优化 v8 跑完，迭代停了下来。这时 Claude 把 v6、v7、v8 共 132 条完整轨迹拆开来看，每一步都有时间戳、工具、参数和结果。下面这些发现决定了后面的方向。

5.1 98% 的时间花在模型"想"上

跑一个工具，中位数 0.06 秒；模型调用一次，中位数 2.7 秒。但模型调用的长尾很重：**超过 20 秒的调用只占 11%，却吃掉了 51% 的模型时间**。每个版本大约有 60 条思考超过 3000 字。最离谱的一次：调用额度已经用完，补丁也是空的，模型又想了 13485 字（约 100 秒），结论只是"我结束了"。

Bo 当时问：这里面是不是也有程序执行的时间，如果是 CPU 在跑，GPU 是不是能腾出来？答：不是。CPU 跑工具一题加起来只有大约 3 秒，瓶颈完全在 GPU 生成文字上。

5.2 改代码的工具 29% 的调用是坏的

`edit_file` 要传两段代码：原来那段（old_string）和新的那段（new_string）。393 次调用里，**112 次报"缺少 old_string"**。原因如下。模型调用工具时，每个参数要写成 `名字:<|"|>值<|"|>`，其中 `<|"|>` 是一个专用的引号符号。模型按字母顺序先写 new_string，但结尾写成了普通的反引号 `` ` ``，于是解析器把后面的 old_string 也吞进了 new_string：

```
new_string:<|"|>self.external_docs = external_docs` ,old_string:<|"|>旧代码<|"|>
                                      ↑ 这里本该是 <|"|>，写成了反引号
```

只传一段文字的 `run_command` 调用了 1663 次，一次都没坏。坏调用集中在少数几道题里，模型会反复重试：fastapi_13713 一题就重试了 21 次。

5.3 读文件只读到开头

25% 的 `read_file` 调用没有传行号。harness 的规则是每次最多返回 150 行，不传行号就只给文件开头。模型拿到的是 import 和文件头，找不到目标，就再读一次，又是开头。日志里能看到模型自己说"I keep reading the beginning of the file"。

Bo 问："read_file 有行号参数，为什么不加？是 Gemma 不加？" 是的。`agent.yaml` 只决定"有哪些工具"，每次调用传什么参数是模型自己决定的。它不加行号，多半是因为它还不知道目标在第几行。

5.4 30 次调用比 5 分钟先用完

这里说的"调用"，是指 agent 调用 harness 提供的工具。模型每次决定跑一条命令、读一个文件、改一处代码，都算一次。30 是我们在 `eval_config.yaml` 里给 v7 设的上限（`max_tool_calls`），由 harness 计数，用满就强制结束这道题。`submit_patch` 和 `get_status` 不计数。

v7 有 5 道题在大约 160 秒就把 30 次调用用光了，一行都没改，交了空补丁，其实还剩 2 分多钟。约 80% 的调用花在 grep 和读文件上，第一次改代码的中位时间是 130 到 150 秒。

5.5 其他问题

- **重复调用**：同一条命令最多重复执行了 18 次。
- **临时文件进了补丁**：调试脚本留在仓库里，被一起交了上去。
- **有些题靠改流程拿不下来**：fastapi_14479 的测试要匹配一条题目里没给出的报错原文；fastapi_14953 需要一个题目里没提到的新文件。

5.6 代码图工具：是什么，为什么这一轮没用上

"代码图"是主办方事先给每个仓库建好的一张"谁和谁有关系"的图（NetworkX 格式）：

- **点**：仓库里的每个函数、类、模块。
- **边**：谁调用谁（CALLS）、谁定义在哪个模块里（DEFINED_IN）、谁 import 了谁（IMPORTS）。
- 每个点还带一个 256 维的 embedding 向量，用来找"长得像"的代码。

工具	做什么
`get_code_neighbors(名字)`	找上下游：谁调用了这个函数，它又调用了谁
`search_similar_code(名字)`	用 embedding 找相似的函数。只能传符号名，不能传一句话
`get_code_subgraph([名字…])`	把几个函数之间的关系一次拿出来

这几个工具不是必须用的：带哪些工具由我们自己决定，v9 就没带。**理论上的好处**是一次就能顺着调用链找上下游，不用一层一层 grep。这可能对"改错文件"那类题有用，因为真正该改的地方常常在调用链的上一层或下一层。

实际情况：46 道题里模型一共只用了 1 到 3 次，有一次还返回空；v14 把它们加回去，一次都没被调用，分数也没涨。它能做的事，`swe refs`、`swe def`、`swe locate`（见第 6 节）基本都能做，而且读的是当前代码。

> **Bo 的判断** 复杂的题、找不对或者找不到的时候，代码图未必没用。这一轮的数据只说明"模型不会主动去用它"，不能说明"它没用"。所以下一轮会把它做成一个兜底手段（见第 11 节）。

6 Bo 的方向：CPU 做查找和排序，LLM 只做判断

6.1 原话

Bo 定的

> "写的时候，可以生成一些 API 函数，让 Gemma 必要的时候调用。比如 grep 这样的子程序：很多任务是重复的，能抽象出来让 CPU 做的（尤其是排序和查找）就交给 CPU；scaffold agent 里形式化的排序、查找，不要用 LLM 的推理去做……尽量让这 129 个 case 被更好地解决。Gemma 本身能力有限，你强，你给它接口函数和知识，把 scaffold 定好。"

> "注意：scaffold 的技巧不能过拟合这 129 题，因为最后是隐藏测试集，必须泛化。"

还有一个关键判断："搜索最强 scaffold 这种事，还是交给 AI 吧，类似 OpenAI 搜索证明未解的数学问题那样。之后我只需要明白最强的 scaffold 为什么强。"

为什么这个方向对：第 5 节已经说明，花时间的是"模型想 + 来回轮数"，不是 CPU。grep 一次只要 0.06 秒，但每做一次 grep，模型都要先想一轮、再发一次调用，一轮就是 3 到 8 秒。**省时间的关键是减少轮数**：让一次 CPU 调用把原来七八轮的活一次做完。

6.2 swe 是什么：不是 harness 工具，是我们自己带进去的命令

harness 的工具是主办方写死的那 9 个，我们没法新增。`swe` 是放在提交包里的一个 **skill 脚本**，就是一个普通的 Python 文件 `skills/swe/scripts/swe.py`。比赛允许提交包带 skill 脚本，它运行在 agent 所在的容器里，看得到仓库代码。接入方式如下：

1. agent 第一次调用 `run_skill_script`（skill 机制自带的工具）时，脚本把自己安装成容器里的一个命令 `swe`；
2. 之后 agent 用 harness 原有的 `run_command` 执行 `swe locate ...`、`swe edit ...`，就像执行 `grep`、`ls` 一样。

所以 harness 一点没改。我们只是借了它原有的"能跑命令"的工具，往容器里加了一个更好用的命令。从模型的角度看，就是多了一个很能干的命令行工具。这也相当于**绕开了主办方的代码图，换了一个更直接的办法**：不用事先建好的图，而是当场扫描当前代码。两者不完全等价：代码图可以顺着调用关系往上下游走好几层，`swe refs` 只能查一层。

6.3 swe 工具箱：给 Gemma 的 CPU 接口

Claude 的优化 Claude 设计了一套命令，**Chrome 执行** Chrome 用 Python 标准库实现了它，名叫 `swe.py`，在 4 个 Python 版本（3.10 到 3.13）的沙箱镜像里跑单元测试，47/47 通过。每条命令对应第 5 节的一个问题：

命令	做什么	解决的问题
`locate 词…`	输入几个题目里的关键词，返回最可能要改的 10 处位置：文件、行号范围、函数名、命中了哪些词、最相关的那一行代码	找代码要七八轮（5.4）
`outline FILE`	列出文件里每个类和函数的行号范围，相当于一张地图	只读到文件头（5.3）
`show FILE 符号/行号`	按函数名或行号显示代码，带行号，最多 120 行	只读到文件头（5.3）
`def` / `refs`	全仓库找一个名字的定义，或者找谁调用了它，按所在函数分组	来回 grep
`edit FILE`	通过 heredoc 传入"查找原文 → 替换成新文"的块。找不到原文时给出最接近的那段；改完自动检查语法，出错就自动退回原样	edit_file 坏 29%（5.2）
`insert` / `replace` / `new`	按行插入、按行替换、新建文件，同样做语法检查和回滚，拒绝写测试文件	需要新建文件的题
`tests` / `test`	找出相关的测试；限时运行，只输出压缩过的失败摘要	验证要便宜（4.3 的教训）
`check`	提交前的一站式检查：还原被改的测试文件、删掉临时文件、检查语法、显示 diff、空补丁时报警	测试文件被改、临时文件进补丁（5.5）
每次输出末尾	`[swe #序号 t=秒]`：显示用了多少时间；过了时限就提示去 check 和提交	模型不知道时间（v4 的发现）
重复刹车	参数、输入和仓库状态都相同的重复调用，在输出开头警告"和第 k 次一样，换个做法"	重复调用（5.5）

改代码为什么用"查找原文 → 替换成新文"的块：

```
swe edit pkg/mod.py <<'EOF'
<<<<<<< SEARCH
原代码（几行，要唯一）
=======
新代码
>>>>>>> REPLACE
EOF
```

- 整段只作为一个字符串参数，通过 `run_command` 传进去。这条路调用 1663 次从没坏过，绕开了 5.2 的问题。
- 这种格式很多模型都见过。引号要写成 `'EOF'`，这样 shell 不会去改动代码里的 `$` 和反引号。
- 匹配分三级：先精确匹配，再忽略行尾空格，再忽略缩进（并自动重新缩进）。匹配到多处时，报出各处行号，要求多给一点上下文。

6.4 针对"30 次调用先用完"：三项改法

#	改法	为什么有用
①	`swe locate` 一次完成 grep 和排序，直接给出最可能要改的文件、行号范围和函数	原来要好几轮 grep、读好几次文件才能找到的位置，一次调用就拿到了
②	`swe outline` 列出文件里的函数和行号，`swe show` 按函数名或行号显示代码	不再出现"读到文件头、找不到、再读一遍"
③	调用上限从 30 提到 40	每次调用干的活变多了，真正的硬约束是时间，多留一点余量

效果：第一次改代码的中位时间从 131 秒提前到 100 秒；调用次数用完、交空补丁的题变少了。

6.5 locate 是怎么排序的（大白话）

说白了，locate 就是一个**专门搜代码的小搜索引擎**，这正是 Bo 说的"排序和查找交给 CPU"。原理和 BM25 给网页排序差不多：

1. 用 Python 自带的 ast 把仓库里所有非测试代码切成一个个函数和类。
2. 每个关键词按"稀有程度"给权重（IDF）：`self`、`return` 到处都有，几乎不加分；`external_docs` 只在两三个地方出现，加很多分。
3. 每个函数的得分 = 它命中的关键词的权重之和。函数名本身命中、或者命中了题目里的一整段报错原文，额外加分。按函数长短做归一化，防止大函数仅仅因为字多就排在前面。
4. 包以外的目录（例子、文档）降低权重，最后输出前 10 个函数，再加一行"最相关的文件"。

它和普通搜索引擎有三点不同：① **一个函数就是一篇"文档"**，所以返回的是"哪个函数"，而不是"哪个文件里出现了这个词"；② **每次都当场重建索引**，仓库不大，扫一遍不到一秒，而且搜的永远是改过之后的最新代码；③ **只按字面匹配，不理解语义**，题目里的词必须真的出现在代码里才能搜到。如果题目说的是"上传文件"，代码里写的是 `UploadFile`，就只能靠模型自己换成正确的词。第 ③ 点是下一步的改进方向（见第 11 节）。

输出大概是这样（只是示意格式）：

```
fastapi/dependencies/utils.py:850-905  request_body_to_args  [File Form UploadFile]  if
isinstance(...
fastapi/params.py:700-786              File.__init__         [File Form]  ...
```

离线检验：在 129 题上，从题目描述里自动抽关键词去跑 locate，再看标准答案改过的文件有没有出现在前 5 名里。开发集上 **file@5 = 0.84**，也就是 84% 的题，要改的文件在前 5 名之内。这一步只用 CPU，不花 GPU。

6.6 skill 脚本在哪里跑：先做实验再定方案

提交物允许带 `skills/` 目录和 .py 脚本，但 README 没写脚本在哪里执行。如果在宿主机上跑，就看不到仓库，工具箱就没用了。所以 Claude 让 Chrome 先做一个探针实验：一个最小的 skill，打印自己所在的机器名、工作目录、能不能看到 `/workspace`。

结论是"A'"：`run_skill_script` 确实在 agent 容器里运行，但每次都放在一个用完就删的临时目录里。办法是：swe 第一次运行时，把自己安装到 `/usr/local/bin`，之后都直接用 `run_command` 调用 `swe ...`。每次 `run_skill_script` 也算一次工具调用。

> **教训** 文档没写清楚的机制，先花 10 分钟做个探针实验，比猜对后面的整个设计重要得多。为了防止 skill 在 Kaggle 上跑不起来，提示词里还准备了退路：skill 用不了时，改用 grep/sed 这些普通命令完成同样的事。

6.7 提示词重写的要点

- **改正一个错误说法。**v7 的提示词写着"不调用 submit 就全部作废"，这是错的：harness 在超时、调用次数用完时，也会把工作区的改动当成补丁去判分。这句话会让模型慌，所以改成"改动无论如何都会判分，先改上去，再逐步完善"。
- **推荐流程。**① 第一次调用就用 locate，传 3 到 8 个题目里的词；② 用 1 到 3 次 show、def、refs 看代码；③ 用 swe edit 改，最迟大约 120 秒或第 12 次调用时要有第一个修改；④ 时间够就做一次便宜的验证；⑤ 运行 check，然后提交，说一句话结束。
- **通用的正确性知识。**题目常常是一个 PR 描述，要把里面列出的每一项都实现。题目里出现的名字和报错文字要逐字照抄。风格模仿相邻代码。改动要最小，并且向后兼容。新的公开名字要和兄弟模块在同一个地方导出。题目提到示例时，要把示例文件一起建出来。
- **约束。**提示词里不能出现任何仓库名、题号，也不能有 `{标识符}` 这种花括号：ADK 会把它当成变量去填，找不到就直接崩溃。

6.8 配置参数

参数	v7	v9	为什么
工具列表	9 个原生工具	`run_command`、`get_status`、`submit_patch` 和 skill	edit_file 坏 29%，read_file 不带行号就读文件头，图工具没人用，所以强制走 swe
thinking_budget	4096	1024	针对 5.1 的长尾
max_output_tokens	8192	6144	思考变短，不需要那么多
max_tool_calls	30	40	每次调用更值钱了，真正的硬约束是时间
timeout_seconds	默认 300	90	防止单条命令（比如误跑了全量测试）吃掉整个预算

7 通宵自动搜索：v9–v14

7.1 规矩

> **Bo 定的**
>
> - Chrome 开最大思考强度，自己设计、实现、跑实验、判断，跑一整晚。
> - **防过拟合**：只在 43 题开发集上迭代；剩下 86 题是留出集，迭代时不准看，最后只用来检验泛化。规则和脚本里不能出现仓库名、题号，也不能有为某一道题写的专用规则。
> - Bo 只看最后的结论，以及"为什么强"。

Claude 写给 Chrome 的指令里，把前面所有的事实、数据和约束都交代清楚了，并要求每个版本都报告一组机制指标：首次编辑时间、空补丁数、编辑失败率、重复调用数、各条 swe 命令的使用次数。不能只报一个总分。

7.2 结果

版本	改了什么	开发集 /43	备注
v7	之前最好的版本	17	
v9	swe 工具箱 + 重写的提示词 + 6.8 的配置	**18**	留出集 40/86，**全量 58/129**，平均每题 3.8 分钟
v10	针对"只读不改"循环做了工具上的修补	16	
v11	关掉 thinking	13	快了 2.7 倍，但少解 3 到 5 题
v12	调用上限改回 30	16	不省时间：耗时长的题是被时限截断的，不是被调用次数
v13	三个编辑和检查方面的修复	15	修复都生效了，丢的是本来就时过时不过的题
v14	加回 v7 的 6 个原生工具	16	write_file 的调用有一半被拒，都是想往 /tmp 写临时文件。原因不是系统权限，而是 harness 的 write_file 只允许写 /workspace 里面的路径。/tmp 本身可以写，但要用 run_command 加 heredoc 来写。v7 的提示词写过这一条，模型还是照老习惯用 write_file，白白浪费了调用次数。图工具一次都没被调用

开发集跑了 8 次，单次分数的标准差约 1.6 题，两个版本相差约 2.2 题以内就分不出好坏。v9 的 18 题属于运气偏好的那一边。

7.3 机制变好了，分数没怎么动

指标	v7	v9
edit 坏调用（缺参数）	约 29%	0
重复调用（开发集合计）	58	36
第一次改代码（中位）	131 秒	100 秒
超过 4000 字的思考	40 条	2 条（说明 thinking 上限确实生效了）
全量	估计 58 ± 5	58

Claude 的判断 效率上的问题基本都解决了，分数却还在原地。说明现在挡住我们的是**模型理解错、改错**，而不是效率。另一个证据：关掉 thinking 会掉 3 到 5 题，说明分数靠的是模型自己的推理能力。再往 scaffold 上加规则，收益会很小。

回头看，这一轮几乎所有优化都集中在**定位和操作**这一层：找得快、读得准、改得稳、交得干净。至于**把 bug 改对**（理解题意、写对逻辑、不漏要求），几乎没有专门的设计。剩下的错题大多出在这一层，所以效率指标明显变好了，分数却没动。开发集 43 题仍然错 25 题，比一半还多。

8 这个分数可信吗：给 v9 的 58/129 做体检

Bo 问："有什么办法可以在 ipython 里检测下这个结果如何，还有可信不？" Claude 写了一个只读的检查脚本，结果如下。

检查	结果	说明
95% 置信区间	36.6% 到 53.6%，约 47 到 69 题	58 应该看成一个范围，不是一个精确的数
补丁和标准答案逐行相同	**19 题**	几乎可以肯定模型背过这些公开仓库的答案。扣掉之后约 39/129 ≈ 30%
超过 300 秒的题	38 题，最长 405 秒	记录的耗时包含判分时间。agent 本身最长 300.7 秒，时限是严格执行的
所有版本做对过的题取并集	72/129	比 v9 多 14 题：这些题模型有时能做对，是 RL 最适合练的
补丁里有临时文件	fastapi_5077 带进了 module_a.py、module_b.py、repro.py、test_sig.py	那次运行在 287 秒被截断，从头到尾没调用 check

Claude 的错误

1. 检查脚本第一版有两个 bug。pytest 的汇总行在安静模式下不带 `===`，正则没匹配上；标准答案补丁的格式和假设的不一样，导致"改的文件和答案有没有重合"那一项把 58 题全部报成了"没重合"。Bo 看到这个结果就追问了，Claude 才发现是脚本的问题。
2. Claude 把 `fastapi/temp_pydantic_v1_params.py` 当成了 agent 留下的临时文件。Chrome 查证后发现，这是仓库里本来就有的文件，标准答案也改了它。
3. Claude 说 fastapi_5077 "改了测试文件"。实际上那是新建在根目录的 test_sig.py，不是改了已有的测试文件。

教训 检查脚本自己也要被检查。只要出现"58 题全部异常"这种结果，先怀疑脚本，再怀疑数据。

9 打包、算时间、提交 Kaggle

9.1 每题给几分钟：推导

Chrome 查到的情况（来自比赛页面和工作人员在讨论区的回答）：隐藏集约 120 题，**一题一题顺序跑**；12 小时包含容器准备时间，不包含判分；超过 12 小时，整个提交就报错。

组成	平均	最长
容器启动 + 解压仓库 + 装 wheel	3.5 秒	6.5 秒
agent	205 秒（5 分钟时限下）	300.7 秒
提取补丁 + 关容器	5.1 秒	约 41 秒
Kaggle 上每题的额外开销（根据官方 notebook 反推）	13 到 16 秒	按 44 秒算

最坏情况：120 × (240 + 16) 秒 ≈ 8.6 小时；每题开销都按 44 秒算是 9.5 小时；再加上装环境、加载模型约 13 分钟，合计 8.8 到 9.7 小时。如果用 5 分钟，最坏约 10.8 小时，留不出 20% 的余量。所以选 **4 分钟**。代价是最多丢掉约 8/58 道本来做得出来的题。

9.2 三个提交版本

版本	改了什么	验证
v9k	v9 只改了一处：时限改成 4 分钟	静态检查（扩展名、符号链接、花括号、仓库名、题号）、主办方的编译器、zip 结构、冒烟测试 3 题全对
v9k2	改进 check 的清理规则：删除根目录下新建的文件，以及名字带 repro、temp、tmp 等的文件。并加了一条兜底：过了时限后，任何 swe 命令都会顺手清理一次	在 129 份补丁上离线检验：会删掉 5 个文件，**标准答案新建的 49 个文件一个都不会误删**；用 5077 的工作区重放验证；新加 17 个单元测试；实跑 5 题全对
v9k3	提示词里的"约 5 分钟"改成 4 分钟；TIME LOW 提示从 200 秒提前到 160 秒，按原来的比例换算	5 题第一遍过 4 题。没过的 5077 补跑 4 次：v9k2 和 v9k3 都是 3 次里过 2 次，属于时过时不过的题，不是改动造成的

> **Bo 定的** 改清理规则可以用标准答案来**验证**它会不会误删，但标准答案不能进入任何提交物。v9k3 判断为"随机波动，可以交"，由 Bo 拍板提交。

9.3 提交流程

1. 把 zip 上传成一个 Kaggle Private dataset。
2. 复制官方的 Getting Started notebook，Add Input 加上这个 dataset；GPU 选 L4 × 4，关掉网络。
3. 第 2 节改成用我们的文件。**一定要删掉那段改写 sampling.yaml 的代码**，否则 thinking 会被悄悄改回 4096。
4. 保留第 5 节的两道题本地推理，在 Kaggle 环境里确认 skill 能用、看实际耗时。
5. Save & Run All，然后 Submit Prediction。每天只能交 1 次，评分要排队。

9.4 结果：0.13（当时榜首 0.24）

本地 0.45 到了隐藏集只剩 0.13，大约 3/10。别人公开说过的比例大约是 1/5（本地 34.5%，隐藏集 6.9%）。掉这么多有三个原因：

1. **背答案。**本地有 19 题的补丁和标准答案逐行相同。扣掉以后，本地靠真本事大约是 30%。
2. **隐藏题更新、更难。**本地 2026 年的题通过率本来就只有约 30%，隐藏集很可能是模型完全没见过的题。
3. **硬件更慢、时限更短。**4 × L4 加 4 分钟，同样的时间里，能走的步数比本地少。

结论：**本地这 129 题已经测不准隐藏集的水平了。**在上面多做对几题，到 Kaggle 上很可能看不出任何变化。作为第一版，0.13 对比榜首 0.24 是一个合理的起点。

10 合作方式上的几点

1. **三方分工。**Bo 定方向、拍板、review；这个对话里的 Claude 做分析和设计，把指令写成可以直接粘贴的 prompt；Chrome 在机器上执行，并把结果写成报告带回来。这种方式最大的好处是 Bo 不用盯

着机器跑。Chrome 一个通宵就跑完了 6 个版本，外加全量和留出集。

2. **Bo 的 review 抓到的错最多。**rich_3486 的定性、检查脚本"58 题全部异常"、"v3 到 v8 怎么只提升了 3 道"……都是 Bo 追问出来的。Claude 和 Chrome 也会互相纠错，比如临时文件那条误判。
3. **说人话。**Bo 多次要求"说人话""适当抽象""简洁"。所有中间汇报都按"结论 → 证据 → 需要你决定的事"来写。
4. **硬约束要写死在每一份指令里。**不停 vLLM、不删 results、标准答案不进提交物、不写仓库名和题号、faketime 只给一题、测试容器不准联网、Claude Code 的授权要 Bo 亲手点。每一份交给 Chrome 的 prompt 都会重复写一遍。
5. **钱。**机器开着就收费，空转也算。以后的做法是：不跑任务就关机，分析和 debug 用 CPU 机器（判分的第二阶段只需要 CPU），批量任务攒到一起通宵跑。
6. **人要能看懂每一题。**Bo 想自己 debug 每道题为什么没过，所以下一步给 CPU 机器准备了两个工具：`case_view.py`（每题生成一个文件夹：题目、agent 补丁对比标准答案、失败的测试、agent 每一步做了什么、一句话失败原因）和 `retest.sh`（用任意补丁重新跑判分）。

11 现在在哪、接下来

现状：v9k3 已经提交，隐藏集 0.13。本地 129 题是 58/129，其中 19 题疑似是背出来的。所有版本合起来做对过 72 题。scaffold 在效率这一层已经基本榨干了。

> **Bo 定的路线** "之后我们可以再随机挑一些更难的题目（构造新的题目），让 AI 去扩展 scaffold，直到不能再提升为止，然后我们看怎么上 RL 去继续提高 Gemma。"

> **Bo 的判断** 直觉上 scaffold 还没榨干：开发集还错一半多，而且所有版本合起来能做对 72 题，比 v9 多 14 题。下一轮 scaffold 要两个方向一起做：**定位**和**改对**。

11.1 下一轮 scaffold

方向	做什么	为什么
先看错题	逐题看开发集错的 25 题，优先看"有时做对"的 14 题（fastapi_14266、fastapi_14616、httpx_3672、requests_7328、rich_3063、rich_3180、rich_3278、rich_3470、rich_3480、rich_3521、rich_3676、rich_3782、rich_4070、rich_4079）。每题判断是"找错了地方"还是"改错了"	这两类的比例决定下一轮的重点
定位	把代码图做成**兜底**：在提示词里写成条件规则（locate 结果不确定，或者改完复现仍然失败，就用 `get_code_neighbors` 去上下游找）。也可以在 swe 里加一个 `swe chain 名字`，用 ast 当场把调用链往上、往下各展开 2 到 3 层	Bo：复杂的题、或者找不对的时候需要它。验证方法：在"改错文件"那类题上，对比加之前和加之后找对位置的题数
定位	locate 改成**关键词加语义的混合召回**：现在的字面匹配，加上主办方给的 embedding 做语义匹配	解决"题目里的说法和代码里的名字不一样"的问题
改对	先复现再改：先在临时目录写一个能复现 bug 的小脚本，改完再跑一遍，确认确实修好了	防止"以为修好了，其实没修好"
改对	要求逐条对照：提交前把题目里的每一条要求（新参数、报错原文、新文件、导出位置）列出来，逐条打勾	防止漏掉要求，也防止报错文字写得不一样
操作	在 `/workspace` 里建一个临时目录（比如 `/workspace/.swe_tmp/`），swe 第一次运行时自动把它写进 `.git/info/exclude`，这样它不会进补丁	Bo 定的。模型习惯在仓库里写临时文件，给它一个合法的地方，比禁止它更有效

11.2 更长期

#	做什么	为什么
1	造新题：从别的 Python 仓库里挑 Gemma 训练截止之后才合并的 PR，每题用标准答案验证环境能跑通（第 2 节的方法写成脚本批量做）	本地分数要能代表隐藏集，背答案的分要排除掉
2	在新题上继续让 AI 扩展 scaffold，直到不再提升	129 题上的提升到隐藏集上看不出来
3	挑出"有时做对"的题：每题采样 4 到 8 次，留下结果有对有错的	GRPO 这类 RL 只能从组内有对有错的题里学到东西
4	把 scaffold 固定住，在上面做 RL 或 LoRA（需要 80G 以上的 GPU）	这样分得清提升来自模型本身，还是来自 scaffold
5	版本比较改成：全量逐题对照，或者同一版本多跑几次	单次开发集的噪声是 ±2 题，分不出差别

待补：CPU 分析的结果（失败题分类、"有时做对"的题清单、NEXT_STEPS.md），以及 Kaggle 上这次提交的实际总运行时长。

附录 A：版本一览

版本	预算	关键变化	开发集 /43	全量 /129
v1	—	主 agent + investigator	—	—
v2	60 分钟 / 100 次	+ Stop clause，临时文件放 /tmp	8	51
v3	5 分钟 / 60 次	只改预算	10	—
v4	同上	时间预算写进提示词	13	—
v5	同上	去掉 investigator	13	—
v6	同上	保护测试文件	17	—
v7	5 分钟 / 30 次	调用上限 30	17	约 58（估计）
v8	同上	强制回归测试	16	—
v9	5 分钟 / 40 次	swe 工具箱、新提示词、thinking 1024	18	58
v10–v14	—	各种修补和消融	13–16	—
v9k / k2 / k3	4 分钟 / 40 次	Kaggle 版、清理规则、时间说法统一	—	Kaggle 0.13

附录 B：关键数字

项目	数字
可以判分的题	129/129（108 题用默认环境 + 21 题单独指定环境）
LLM 时间占比	约 98%；超过 20 秒的调用占 11%，吃掉 51% 的模型时间
edit_file 坏调用	112/393（29%）；run_command 是 0/1663
locate 召回	file@5 = 0.84（开发集）
开发集噪声	单次标准差约 1.6 题
v9 全量	58/129，95% 置信区间 47 到 69 题；19 题与标准答案逐行相同；所有版本合起来做对过 72 题
Kaggle	约 120 题顺序跑，12 小时；4 分钟时限下最坏约 8.8 到 9.7 小时
隐藏集	0.13（当时榜首 0.24）

附录 C：术语

词	大白话
scaffold	模型外面的那一套"脚手架"：提示词、工具、预算、流程。不改模型权重。
harness	主办方的评测程序：负责起容器、跑 agent、取补丁、判分。
gold patch	标准答案补丁，也就是 PR 里真实的改动。
test_patch	判分时额外打上的隐藏测试。
开发集 / 留出集	开发集用来迭代、挑版本；留出集迭代时不看，最后用来检验有没有过拟合。
skill	提交物里可以带的脚本和说明，比如 swe 工具箱。
thinking budget	模型每次在给出回答之前，最多能"想"多少 token。
GRPO	一种 RL 方法：同一题采样多次，做对的那几次加分，做错的减分。

附录 D：链接

内容	链接
比赛页面	https://www.kaggle.com/competitions/gemma-4-developer-agent
Google ADK（agent 框架）	https://github.com/google/adk-python
ADK Skills 文档	https://adk.dev/skills/
vLLM（推理服务）	https://github.com/vllm-project/vllm
SWE-bench（题目形式的来源）	https://github.com/SWE-bench/SWE-bench
4 个题目仓库	https://github.com/fastapi/fastapi · https://github.com/Textualize/rich · https://github.com/psf/requests · https://github.com/encode/httpx
文中例题 fastapi_11194 对应的 PR	https://github.com/fastapi/fastapi/pull/11194
系列博客	https://jsjhoubo.github.io